

Department of Computer Science and Engineering

Faculty of Technology and Engineering,

The Maharaja Sayajirao University of Baroda, Vadodara

51, Kalabhavan, Vadodara - 390001

Hack-A-Throne-25

PS 21:

QA Unit test and test harness creation - Developer/QA

Tool, AI

Organizations often face challenges in achieving faster and reliable software testing due to the manual effort required in writing unit tests, creating QA automation scripts, and managing test data. Current processes are time-consuming, error-prone, and lack scalability, resulting in delayed releases and inconsistent test coverage.

There is a need for a tool that can automatically scan source code and generate unit tests, while also enabling QA automation. The tool should be capable of externalizing test data sets, exporting test cases to external files (e.g., Excel), and then automatically converting these cases into executable automation scripts. Finally, the tool must execute the generated test cases and provide comprehensive test reports to support decision-making and ensure high software quality with minimal manual intervention.

In-Scope

The objective is to develop an intelligent test automation tool that:

- Analyzes source code and automatically generates unit tests and integration tests.
- Creates QA automation scripts by transforming externally defined test cases into executable scripts.
- Externalizes test data to allow easy management, reusability, and scalability.
- Exports and imports test cases through external files (e.g., Excel) for better collaboration between developers, testers, and business teams.
- Executes generated test cases and produces detailed automation reports, enabling teams to validate quality efficiently and accelerate release cycles.

QA Unit test and test harness creation – Developer / QA Tool, AI

Organizations often face challenges in achieving faster and reliable software testing due to the manual effort required in writing unit tests, creating QA automation scripts,

and managing test data. Current processes are time-consuming, error-prone, and lack scalability, resulting in delayed releases and inconsistent test coverage.

There is a need for a tool that can automatically scan source code and generate unit tests, while also enabling QA automation. The tool should be capable of externalizing test data sets, exporting test cases to external files (e.g., Excel), and then automatically converting these cases into executable automation scripts. Finally, the tool must execute the generated test cases and provide comprehensive test reports to support decision-making and ensure high software quality with minimal manual intervention.

In-Scope

The objective is to develop an intelligent test automation tool that:

- Analyzes source code and automatically generates unit tests and integration tests.
- Creates QA automation scripts by transforming externally defined test cases into executable scripts.
- Externalizes test data to allow easy management, reusability, and scalability.
- Exports and imports test cases through external files (e.g., Excel) for better collaboration between developers, testers, and business teams.
- Executes generated test cases and produces detailed automation reports, enabling teams to validate quality efficiently and accelerate release cycles.

"AI-Powered Intelligent Test Automation Toolkit"

• Context:

In modern software development lifecycles, ensuring high-quality releases at speed is a growing challenge. Manual creation of unit tests, test data management, and QA automation consumes valuable time and resources, often leading to inconsistent test coverage and delayed product delivery.

• Problem:

Organizations lack a **scalable, intelligent tool** that can **automatically generate and manage test cases** from source code and external specifications. The current methods are **manual, fragmented, and error-prone**, increasing the time to market and risk of undetected bugs in production.

• **Your Challenge (In-Scope Objectives):**

Your team is tasked with building a **smart developer/QA tool** that addresses these core testing pain points. The solution should aim to:

1. **Auto-generate Unit & Integration Tests**
Analyze the provided source code and intelligently generate relevant unit and integration tests, reducing manual effort for developers.
2. **Transform Test Cases into Automation Scripts**
Enable users to define test cases in external files (like Excel), which your tool will ingest and convert into executable QA automation scripts.
3. **Externalize & Manage Test Data**
Allow easy management of test datasets through external files for reusability and scalability.
4. **Import/Export Test Cases**
Support collaboration by exporting/importing test cases in structured formats (e.g., Excel), making it easier for developers, testers, and business users to contribute.
5. **Execute Tests & Generate Reports**
Automatically run the generated tests and provide a **comprehensive test report** (pass/fail, coverage, logs) to support rapid decision-making and continuous integration.

• **Bonus (for extra points):**

- Integrate AI to intelligently **suggest edge cases** or **identify missing tests**.
- Support **code coverage analysis** or **test impact analysis**.
- Create a **user-friendly dashboard** to view test reports and statistics.
- Add support for **multiple programming languages or frameworks**.

• **Deliverables:**

- A working prototype of the intelligent test automation tool.
- Source code repository with setup instructions.

• Bonus / Extension Tracks (Optional, Additional)

For teams looking to push the boundaries further, consider extending your solution with the following advanced capabilities:

• 1. DevOps Integration

- Integrate your testing tool into a CI/CD pipeline (e.g., GitHub Actions, GitLab CI, Jenkins).
- Enable automated triggering of test generation and execution on pull requests or code merges.
- Allow test reports to be pushed as build artifacts or comments on PRs.

• 2. Continuous Testing Support

- Support **incremental test generation** based on changed code only (test impact analysis).
- Use static analysis or AI to recommend which tests to rerun in large codebases.

• 3. Deployment-Ready Testing Dashboard

- Deploy a **web-based dashboard** to view test reports, coverage trends, and failure insights.
- Provide APIs or webhook integrations to notify stakeholders (Slack, email, etc.).

• 4. Test Coverage & Metrics

- Generate code coverage reports (e.g., line, branch, function) integrated into the dashboard or CI.